

VirtuEx 시스템 감사 보고서 (1차)

프로젝트: VirtuEx - 가상 자산 모의 거래 플랫폼 차수: 1차 감사 작성일: 2026-03-01 작성: 시스템 감사팀

1. 개요

VirtuEx는 NestJS 마이크로서비스 백엔드(8개 서비스)와 Next.js 15 프론트엔드로 구성된 가상 자산 모의 거래 플랫폼입니다. 본 보고서는 시스템 전반에 대한 보안, 성능, 아키텍처 감사 결과를 정리합니다.

1.1 서비스 구성

서비스	포트	역할
API Gateway	3000	프록시, WebSocket, 인증
Market Data	3001	시세 데이터, Binance 연동
Order Engine	3002	주문 처리, 매칭 엔진
Portfolio	3003	자산 관리, 정산
Notification	3004	이메일 알림
Chat	3005	실시간 채팅
AI Service	3006	AI 분석
User Auth	3007	인증, 사용자 관리

2. 보안 감사

2.1 인증 및 권한

- JWT 기반 인증:** Access Token (15분) + Refresh Token (7일, httpOnly Cookie)
- SMS 2단계 인증:** 로그인 시 SMS 코드 필수 검증
- TOTP 지원:** Google Authenticator 등 TOTP 앱 지원
- 역할 기반 접근 제어:** SYSTEM > ADMIN > USER 3단계
- 계정 잠금:** 인증 실패 5회 시 자동 잠금

2.2 수정된 보안 이슈

#	항목	위험도	상태
5	.env.example에 민감값 노출	중	수정 완료 - 플레이스홀더로 교체
12	SMS 코드 프로덕션 로그 노출	상	수정 완료 - production 마스킹
13	하드코딩된 보안 상수	중	수정 완료 - ConfigService 이관
17	Docker 자격증명 하드코딩	중	수정 완료 - 환경변수 패턴
27	로그인 실패 로깅 미흡	하	기 수정 완료

2.3 잔여 권고사항

- Rate Limiting:** API Gateway 레벨 속도 제한 강화 권고
- CORS:** 프로덕션 배포 시 허용 오리진 제한 필수
- Helmet.js:** HTTP 보안 헤더 적용 권고

3. 데이터 무결성

3.1 수정된 항목

#	항목	상태
4	TriggerType enum 미검증	수정 완료 - @IsEnum(TriggerType) 적용
6	FK 인덱스 누락	수정 완료 - Comment.authorId, Participant.roomId 인덱스 추가
9	응답 포맷 불일치	수정 완료 - { success, data } 통일
10	Null 체크 및 select 절 부재	수정 완료 - getOrder/getUserOrders에 select 명시
26	비활성화 시점 미기록	수정 완료 - deactivatedAt 필드 추가

3.2 데이터베이스 구조

- **Database per Service** 패턴: 서비스별 독립 DB (mex_auth, mex_orders 등)
- **Event Sourcing**: Order Engine의 주문 이벤트 불변 저장
- **CQRS**: 쓰기(이벤트스토어) / 읽기(PostgreSQL) 분리

4. 서비스 간 통신

4.1 현재 구조



4.2 현재 완화 조치

- **Retry with Exponential Backoff**: withRetry() (최대 3회, 2^n초 대기)
- **Timeout**: 설정 가능한 HTTP 타임아웃 (INTERNAL_HTTP_TIMEOUT 환경변수)
- **Circuit Breaker**: 미적용 (로드맵 항목)

4.3 비동기 전환 로드맵 (#14)

현재 문제: Order Engine → Portfolio/Market Data 간 HTTP 동기 호출로 인한 결함도

단계별 전환 계획:

1. **Phase 1 (단기)**: 현재 - withRetry + timeout으로 안정성 확보 ✅
2. **Phase 2 (중기)**: Circuit Breaker 패턴 적용 (Kafka Consumer 그룹 활용)
 - 주문 정산을 Kafka 이벤트 기반으로 전환
 - ORDER_FILLED 이벤트 → Portfolio Consumer가 비동기 정산
3. **Phase 3 (장기)**: 완전한 이벤트 기반 아키텍처
 - Saga 패턴으로 분산 트랜잭션 관리
 - 보상 트랜잭션(Compensating Transaction) 구현

5. 인프라 감사

5.1 Docker 구성

항목	상태
PostgreSQL healthcheck	수정 완료 - DB 지정 검사
리소스 제한	수정 완료 - postgres 512M, redis 256M, kafka 1G
자격증명 보안	수정 완료 - 환경변수 패턴
Prometheus 설정	수정 완료 - 전체 8개 서비스 등록
DB 백업	신규 - backup-db.sh (pg_dump + gzip, 7개 보존)

5.2 모니터링 스택

- **Prometheus:** 메트릭 수집 (15초 간격)
- **Grafana:** 대시보드 시각화
- **Jaeger:** 분산 트레이싱 (OpenTelemetry)

5.3 중앙 로깅 로드맵 (#19)

현재: 각 서비스 로컬 파일 로깅 + 콘솔 출력

권고 구성:

서비스	→	Fluentd/Filebeat	→	Elasticsearch	→	Kibana
		(또는 Loki)		(또는 Loki)		(또는 Grafana)

도입 계획:

1. **Phase 1:** 구조화 로깅 (JSON 포맷) 표준화 - LoggingInterceptor 활용 
2. **Phase 2:** Loki + Grafana 스택 도입 (Prometheus와 통합 용이)
3. **Phase 3:** 로그 기반 알림 및 대시보드 구성

6. 프론트엔드 감사

6.1 UX 개선

#	항목	상태
22	주요 페이지 로딩 상태	기 적용 (Skeleton 컴포넌트)
24	모달 모바일 반응형	수정 완료
36	통계 모바일 탭	수정 완료 - overflow-x-auto
37	도움말 모바일 탭	수정 완료 - overflow-x-auto
38	서비스 상태 포트순 정렬	수정 완료
39	햄버거 admin 메뉴	수정 완료 - flex-col
41	NaN 표시 수정	수정 완료 - Number.isFinite 가드

6.2 폼 검증 및 접근성

#	항목	상태
23	aria-label 추가	수정 완료
25	onBlur 검증	수정 완료
42	SMA5 기본 활성화	수정 완료
43	매수 에러 토스트	수정 완료

6.3 상태 관리

- **Zustand**: 클라이언트 상태 (auth, settings, toast) - persist middleware 활용
- **TanStack React Query**: 서버 상태 (API 데이터 캐싱, 자동 갱신)
- **WebSocket**: Socket.io를 통한 실시간 가격/채팅 스트리밍

7. 시스템 설정 및 관리

7.1 신규 기능

#	항목	상태
35	SystemSetting 모델 및 API	신규 구현
34	설정 화면 view/edit 모드	수정 완료
35	설정 API 연동	수정 완료
40	알림 설정 백엔드 연동	수정 완료 (optimistic update)

8. 헬스체크 명령어 (#44)

전체 서비스 상태 확인

```
# API Gateway
curl -s http://localhost:3000/api/health | jq .

# 전체 서비스 상태 (API Gateway 프록시)
for svc in user-auth market-data order-engine portfolio notification chat ai-service; do
  echo "=== $svc ==="
  curl -s "http://localhost:3000/api/health/$svc" | jq .
done

# 서비스 상세 (프로브 + 메트릭)
curl -s http://localhost:3000/api/health/user-auth/detail | jq .

# Docker 인프라 상태
docker compose ps

# PostgreSQL 연결 확인
pg_isready -h localhost -p 5432 -U postgres -d mockexchange

# Redis 상태
redis-cli ping
```

```
# Kafka 상태
docker exec mex-kafka /opt/kafka/bin/kafka-broker-api-versions.sh --bootstrap-server localhost:9092
```

서비스별 직접 확인

```
# 각 서비스 직접 헬스체크
curl -s http://localhost:3000/health/live # API Gateway
curl -s http://localhost:3001/health/live # Market Data
curl -s http://localhost:3002/health/live # Order Engine
curl -s http://localhost:3003/health/live # Portfolio
curl -s http://localhost:3004/health/live # Notification
curl -s http://localhost:3005/health/live # Chat
curl -s http://localhost:3006/health/live # AI Service
curl -s http://localhost:3007/health/live # User Auth
```

9. 종합 평가

9.1 강점

- 마이크로서비스 아키텍처로 서비스 간 독립 배포 가능
- Event Sourcing + CQRS로 주문 이력 완전 추적
- JWT + SMS 2FA로 이중 인증 보안
- 실시간 가격 스트리밍 (WebSocket + Kafka)
- Prisma ORM으로 타입 안전한 DB 접근

9.2 개선 완료 항목 요약

영역	수정 항목 수
백엔드 데이터 무결성	5
보안 및 설정	4
WebSocket/서비스 구조	3
인프라	4
프론트엔드 UX	7
폼 검증/접근성	4
시스템 설정/알림	4
문서/감사	5
합계	36

9.3 향후 로드맵

1. **Circuit Breaker**: 서비스 간 장애 전파 방지
2. **중앙 로깅**: Loki/ELK 스택 도입
3. **비동기 정산**: Kafka 이벤트 기반 포트폴리오 정산
4. **Rate Limiting**: API Gateway 요청 제한 강화
5. **E2E 테스트**: Playwright 기반 통합 테스트 확대